

Hydroponics ESP32 Monitoring System

Project Handover Report

Date: 19 June 2026

Status: Working breadboard prototype (ESP1) — handover for continuation

Companion technical document: `HANDOVER.md` (in the project repository)

Contents

1	Project Overview	2
1.1	System architecture	2
2	Work Completed So Far	2
3	Current Implementation Status	2
4	Hardware and Wiring	3
4.1	Pin map (ESP1)	3
4.2	Power	3
4.3	Voltage dividers (5 V signals into the ESP)	3
5	Software Setup and How to Run	4
5.1	Dependencies to install	4
5.2	Configure the network settings	4
5.3	Build and flash the ESP32	4
5.4	Register devices and send commands	5
5.5	Monitor the live data	5
6	Pending Tasks	5
7	Repository Contents and Attachments	5

1. Project Overview

This project is an Internet-of-Things (IoT) monitoring and control system for a hydroponics installation. Each ESP32 microcontroller board reads a set of environmental and water-quality sensors and drives four relays (for pumps, valves, or lights). Sensor readings are published over MQTT to a backend built on the EdgeX IoT platform, and relay commands are received back over MQTT.

The system is designed to scale to four identical ESP nodes (ESP1–ESP4). To date, one node (ESP1) has been assembled on a breadboard and validated end-to-end.

1.1 System architecture

Component	Location	Role
ESP32 firmware	src/ (C++, PlatformIO)	Reads sensors, drives relays, communicates over WiFi/MQTT/HTTP
Python tooling	Repository (*.py)	Registers/deletes devices, profiles, streams and rules; sends commands
Backend (existing)	EdgeX + Mosquitto brokers on the LAN	Stores device metadata, bridges MQTT data, runs rules (eKuiper)

Data flow: the ESP fetches its assigned device list over HTTP from a configuration server, then publishes each reading over MQTT. The Python scripts register device definitions into the EdgeX backend so that the platform recognises and routes the data.

2. Work Completed So Far

- Full ESP32 firmware (C++/PlatformIO) with a modular, object-oriented driver per sensor type and a runtime device manager.
- Dynamic device provisioning: the firmware fetches its device list from the backend and creates, updates, or removes sensor objects at runtime — no re-flashing needed to add or move a device.
- Reliable connectivity: WiFi and MQTT auto-reconnect, with an automatic ESP restart after repeated failures.
- Complete Python tooling to register and remove devices, profiles, streams, and rules, and to send relay commands, all driven by JSON configuration files.
- ESP1 assembled on a breadboard with seven sensor types plus a four-channel relay module.
- End-to-end validation: live sensor data confirmed flowing from the ESP, through the broker, and visible in MQTT Explorer.
- Two firmware bugs identified and fixed during testing: the MQ-7 analog/digital pins were swapped, and the temperature/humidity sensor was reconfigured for the correct DHT22 type on a stable GPIO.

3. Current Implementation Status

Four of seven sensor types are fully working; the remaining three are limited by hardware (5V power and voltage dividers) rather than software.

Sensor	GPIO	Reading	Status
DHT22 (temp/hum.)	4	28.2 °C / 44%	WORKING — sensible room values
TDS (water)	34	0 ppm (in air)	WORKING — correct dry; rises in water
Ambient light	36	0–4095	WORKING — responds to light level
Relays 1–4	21/18/5/27	off	WORKING — status + command verified
pH (4502C)	35	~9.8	PARTIAL — needs calibration + divider
MQ-7 (gas)	32/33	0	PENDING — needs 5 V heater supply
HC-SR04 (dis-tance)	23/22	invalid	PENDING — needs 5 V + divider on ECHO

Note: sensor polling intervals are currently set to 5 seconds across all drivers to make live testing easy. These should be raised to production values before deployment (see Pending Tasks).

4. Hardware and Wiring

4.1 Pin map (ESP1)

GPIO	Component	Sensor pin	Power	Voltage divider?
4	DHT22	DATA	3.3 V	No (10 kΩ pull-up to 3.3 V)
36	Ambient light V2.1	AOUT	3.3 V	No
32	MQ-7	DOUT	5 V	Yes
33	MQ-7	AOUT	5 V	Yes
34	TDS	AOUT	5 V	Yes
35	pH 4502C	AOUT	5 V	Yes
23	HC-SR04	TRIG	5 V	No (ESP output)
22	HC-SR04	ECHO	5 V	Yes
21/18/5/27	Relays 1–4	IN1–IN4	5 V	No (ESP outputs)

4.2 Power

- Both 3.3 V and 5 V are taken from the ESP32 development board itself when it is powered over USB. The 3V3 pin powers the DHT22 and light sensor; the 5V (VIN) pin powers the MQ-7, TDS, pH, HC-SR04, and relays.
- No separate external power supply is required for the sensors. All components must share a common ground with the ESP.
- When the four relays are connected, a laptop USB port may not supply enough current. Power the relay board from a 5 V phone charger or powered USB hub (still sharing ground with the ESP).

4.3 Voltage dividers (5 V signals into the ESP)

ESP32 input pins tolerate a maximum of 3.3 V. Every 5 V signal that goes *into* the ESP must be reduced with a resistor voltage divider. Signals that *leave* the ESP (TRIG and the relay inputs) do not need one.

Recommended divider using available resistors: two $510\ \Omega$ resistors in series ($1020\ \Omega$) on the signal side, and one $2\ \text{k}\Omega$ resistor to ground. This produces $5\ \text{V} \times 2000/3020 \approx 3.31\ \text{V}$, which is safe.

A single $510\ \Omega$ resistor on top must NOT be used — it yields $3.98\ \text{V}$, which exceeds the safe limit.

Dividers are required on five inputs: GPIO 22 (ECHO), 32 (MQ-7 DOUT), 33 (MQ-7 AOUT), 34 (TDS), and 35 (pH). That is ten $510\ \Omega$ resistors and five $2\ \text{k}\Omega$ resistors per ESP.

Calibration: the divider feeds the ESP about 0.66 of the true voltage, so multiply the measured voltage by 1.51 in the pH and TDS drivers to recover real values (not yet applied).

5. Software Setup and How to Run

5.1 Dependencies to install

- Visual Studio Code with the PlatformIO IDE extension (or PlatformIO Core CLI: `pip install platformio`).
- The ESP libraries are installed automatically by PlatformIO from `platformio.ini`: `ArduinoJson`, `PubSubClient`, `Adafruit Unified Sensor`, `DHT sensor library`, `ezButton`, `MQUnifiedsensor`, `DallasTemperature`, `NTPClient`.
- Python 3, plus the MQTT client library: `pip install paho-mqtt`.
- MQTT Explorer (desktop app) for live monitoring; optionally `mosquitto-clients` for command-line monitoring.
- A reachable backend: the EdgeX stack and Mosquitto brokers on the LAN (pre-existing infrastructure).

5.2 Configure the network settings

These values are hardcoded and almost certainly must be changed for a new network:

- In `src/main.cpp`: WiFi SSID and password, the MQTT broker IP (`mqtt_server`), the configuration-server URL (`devices_url`), and the node identity (`esp_name = ESP1.ESP4`).
- In `variables.json`: `mqttBroker` (the broker the Python scripts publish to) and the EdgeX service IPs.

Important: the firmware and the Python scripts currently point at two different brokers (the ESP uses 192.168.10.102; the scripts use 192.168.10.103). This reflects the existing backend layout. Confirm the correct addresses with whoever maintains the EdgeX deployment.

5.3 Build and flash the ESP32

1. Edit `src/main.cpp` with the correct WiFi and broker settings.
2. Connect the ESP32 by USB (install the CP210x or CH340 USB driver if no COM port appears).
3. In the PlatformIO sidebar, choose the `upezzy_wroom` environment and click “Upload and Monitor” (CLI: `pio run -t upload`, then `pio device monitor`).
4. Confirm the serial output shows: WiFi connected, MQTT connected, the device list, and live readings.

Project structure note: the firmware entry point is `src/main.cpp`. An earlier `src/main/main.ino` was renamed to `src/main.cpp` because PlatformIO only compiles `.cpp` files located directly inside `src/`.

5.4 Register devices and send commands

1. `python add_profile.py` — pushes the device profiles from `device_profile.json`.
2. `python add_device_from_json.py` — registers every device from `devices_list.json`.
3. To change a wiring/GPIO and re-register: `python delete_all_devices.py`, then `python add_device_from_json.py`.
4. `python command_device.py` — sends a relay command (edit the `Command` block in `variables.json` first).

5.5 Monitor the live data

- Open MQTT Explorer and connect to the ESP broker (192.168.10.102, port 31883, no TLS, no authentication).
- Expand `incoming` → `data`. Each device appears as its own branch with values that update in place, e.g. `dht-esp1-.../temperature`, `tds-esp1-.../TDS value`, `phx-esp1-.../Ph value`.
- Command-line alternative for one sensor: `mosquitto_sub -h 192.168.10.102 -p 31883 -t 'incoming/data/phx-esp1-#' -v`

6. Pending Tasks

1. Build and install the five voltage dividers (GPIO 22, 32, 33, 34, 35) for safe, accurate analog readings.
2. Power the MQ-7, HC-SR04, TDS, and pH sensors from 5 V (the VIN pin), not 3.3 V.
3. Apply the $\times 1.51$ divider correction in `phSensor.cpp` and `tdsSensor.cpp` once dividers are fitted.
4. Calibrate the pH sensor (pH 4 and pH 7 buffer solutions) and the TDS sensor (known reference solution).
5. Restore production polling intervals — every sensor `loop()` is currently 5000 ms for testing.
6. Wire and test the four relays, powering the relay board from a dedicated 5 V supply.
7. Replicate ESP1 onto ESP2–ESP4 by changing `esp_name` and re-flashing (devices are already defined).
8. Consider unifying the two MQTT brokers if the backend is redeployed.

7. Repository Contents and Attachments

The complete source code, configuration, and the companion technical document (`HANDOVER.md`) accompany this report. Key items:

Path	Description
src/	ESP32 firmware: main.cpp, device manager/factory, and one driver per sensor and the relay actuator
platformio.ini	Board, framework, and library definitions
devices_list.json	All devices for ESP1–ESP4 (name, profile, GPIOs)
device_profile.json	EdgeX device profiles (resources, value types, units)
variables.json	Broker IPs, MQTT topic names, and script payloads
*.py	Python tooling: add/delete devices, profiles, streams, rules; command sender; device simulator
HANDOVER.md	Full technical handover (wiring, dependencies, run steps, file reference)
hydroponics_setting_up.	Original setup guide from the previous team